```
"""
Data Preprocessing Techniques on a Real-World Dataset
=====================================================
Dataset: Titanic (loaded via seaborn's built-in dataset repository)

Techniques demonstrated:
1. Initial data inspection
2. Handling missing values
3. Normalization (Min-Max Scaling)
4. Standardization (Z-score Scaling)
5. Encoding techniques:
  - Label Encoding
  - One-Hot Encoding
  - Ordinal Encoding
6. Putting it together with a ColumnTransformer / Pipeline
"""

import pandas as pd
import numpy as np
import seaborn as sns

from sklearn.preprocessing import (
  MinMaxScaler,
  StandardScaler,
  LabelEncoder,
  OneHotEncoder,
  OrdinalEncoder,
)
```

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer

pd.set_option("display.max_columns", None)
pd.set_option("display.width", 120)



# ---------------------------------------------------------------------------
# 1. LOAD THE DATASET
# ---------------------------------------------------------------------------
def load_data():
    df = sns.load_dataset("titanic")
    print("Original shape:", df.shape)
    print("\nFirst 5 rows:\n", df.head())
    print("\nData types:\n", df.dtypes)
    print("\nMissing values per column:\n", df.isnull().sum())
    return df



# ---------------------------------------------------------------------------
# 2. HANDLE MISSING VALUES
# ---------------------------------------------------------------------------
def handle_missing_values(df):
    df = df.copy()

    # Drop columns that are mostly empty or redundant for this demo
    df.drop(columns=["deck", "embark_town", "alive"], inplace=True, errors="ignore")
```

```
    # Numerical columns -> impute with median
    num_imputer = SimpleImputer(strategy="median")
    df["age"] = num_imputer.fit_transform(df[["age"]])

    # Categorical columns -> impute with most frequent value
    cat_imputer = SimpleImputer(strategy="most_frequent")
    df["embarked"] = cat_imputer.fit_transform(df[["embarked"]]).ravel()

    print("\nMissing values after imputation:\n", df.isnull().sum())
    return df


# ---------------------------------------------------------------------
# 3. NORMALIZATION (Min-Max Scaling -> scales values into [0, 1])
# ---------------------------------------------------------------------
def normalize_features(df, columns):
    df = df.copy()
    scaler = MinMaxScaler()
    df[[f"{c}_norm" for c in columns]] = scaler.fit_transform(df[columns])
    print(f"\nNormalized columns {columns} (Min-Max Scaling):")
    print(df[[f"{c}_norm" for c in columns]].describe())
    return df, scaler


# ---------------------------------------------------------------------
# 4. STANDARDIZATION (Z-score Scaling -> mean 0, std 1)
# ---------------------------------------------------------------------
```

```
def standardize_features(df, columns):
    df = df.copy()
    scaler = StandardScaler()
    df[[f"{c}_std" for c in columns]] = scaler.fit_transform(df[columns])
    print(f"\nStandardized columns {columns} (Z-score Scaling):")
    print(df[[f"{c}_std" for c in columns]].describe())
    return df, scaler



# ---------------------------------------------------------------------------
# 5. ENCODING TECHNIQUES
# ---------------------------------------------------------------------------
def encode_features(df):
    df = df.copy()

    # -- Label Encoding: for a binary / ordinal-like categorical column --
    le = LabelEncoder()
    df["sex_label"] = le.fit_transform(df["sex"])
    print("\nLabel Encoding on 'sex':", dict(zip(le.classes_, le.transform(le.classes_))))

    # -- Ordinal Encoding: when categories have a natural order --
    class_order = [["Third", "Second", "First"]]
    ordinal_enc = OrdinalEncoder(categories=class_order)
    df["pclass_name_ordinal"] = ordinal_enc.fit_transform(df[["class"]])
    print("\nOrdinal Encoding on 'class' (Third < Second < First):")
    print(df[["class", "pclass_name_ordinal"]].drop_duplicates())

    # -- One-Hot Encoding: for nominal categories with no order --
```

```
    ohe = OneHotEncoder(sparse_output=False, drop="first")
    embarked_ohe = ohe.fit_transform(df[["embarked"]])
    ohe_cols = ohe.get_feature_names_out(["embarked"])
    df_ohe = pd.DataFrame(embarked_ohe, columns=ohe_cols, index=df.index)
    df = pd.concat([df, df_ohe], axis=1)
    print("\nOne-Hot Encoding on 'embarked':\n", df[ohe_cols].head())

    return df


# ---------------------------------------------------------------------------
# 6. FULL PIPELINE (combining imputation, scaling, and encoding)
# ---------------------------------------------------------------------------
def build_preprocessing_pipeline():
    numeric_features = ["age", "fare", "sibsp", "parch"]
    categorical_features = ["sex", "embarked", "class"]

    numeric_transformer = Pipeline(steps=[
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler()),
    ])

    categorical_transformer = Pipeline(steps=[
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("onehot", OneHotEncoder(handle_unknown="ignore")),
    ])

    preprocessor = ColumnTransformer(transformers=[
```

```
        ("num", numeric_transformer, numeric_features),
        ("cat", categorical_transformer, categorical_features),
    ])

    return preprocessor, numeric_features, categorical_features



def run_full_pipeline(df):
    preprocessor, num_feats, cat_feats = build_preprocessing_pipeline()
    X = df[num_feats + cat_feats]

    transformed = preprocessor.fit_transform(X)

    # Recover readable column names
    cat_ohe_cols =
preprocessor.named_transformers_["cat"]["onehot"].get_feature_names_out(cat_feats)
    all_cols = num_feats + list(cat_ohe_cols)

    transformed_df = pd.DataFrame(
        transformed.toarray() if hasattr(transformed, "toarray") else transformed,
        columns=all_cols,
    )
    print("\nFinal preprocessed feature matrix (via Pipeline + ColumnTransformer):")
    print(transformed_df.head())
    return transformed_df
```

```
# ----------------------------------------------------------------------

# MAIN

# ----------------------------------------------------------------------

if __name__ == "__main__":

    df = load_data()

    df = handle_missing_values(df)


    df, minmax_scaler = normalize_features(df, ["age", "fare"])

    df, std_scaler = standardize_features(df, ["age", "fare"])


    df = encode_features(df)


    print("\n" + "=" * 70)

    print("Combined preprocessing pipeline (recommended approach for ML models)")

    print("=" * 70)

    final_df = run_full_pipeline(df)


    print("\nAll preprocessing steps completed successfully.")
```

```
Original shape: (891, 15)

First 5 rows:
    survived  pclass     sex   age  sibsp  parch     fare embarked  class
who  adult_male deck  embark_town alive  \
0         0       3    male  22.0      1      0   7.2500        S  Third
man        True  NaN  Southampton    no
1         1       1  female  38.0      1      0  71.2833        C  First
woman       False    C    Cherbourg   yes
2         1       3  female  26.0      0      0   7.9250        S  Third
woman       False  NaN  Southampton   yes
3         1       1  female  35.0      1      0  53.1000        S  First
woman       False    C  Southampton   yes
4         0       3    male  35.0      0      0   8.0500        S  Third
man        True  NaN  Southampton    no

   alone
```

```
0  False
1  False
2   True
3  False
4   True

Data types:
 survived          int64
pclass            int64
sex              object
age             float64
sibsp             int64
parch             int64
fare            float64
embarked         object
class          category
who              object
adult_male         bool
deck           category
embark_town      object
alive            object
alone              bool
dtype: object

Missing values per column:
 survived         0
pclass           0
sex              0
age            177
sibsp            0
parch            0
fare             0
embarked         2
class            0
who              0
adult_male       0
deck           688
embark_town      2
alive            0
alone            0
dtype: int64

Missing values after imputation:
 survived      0
pclass        0
sex           0
age           0
sibsp         0
parch         0
fare          0
embarked      0
class         0
who           0
adult_male    0
alone         0
dtype: int64

Normalized columns ['age', 'fare'] (Min-Max Scaling):
         age_norm   fare_norm
count  891.000000  891.000000
mean     0.363679    0.062858
```

```
std      0.163605    0.096995
min      0.000000    0.000000
25%      0.271174    0.015440
50%      0.346569    0.028213
75%      0.434531    0.060508
max      1.000000    1.000000

Standardized columns ['age', 'fare'] (Z-score Scaling):
            age_std      fare_std
count  8.910000e+02  8.910000e+02
mean   2.272780e-16  3.987333e-18
std    1.000562e+00  1.000562e+00
min   -2.224156e+00 -6.484217e-01
25%   -5.657365e-01 -4.891482e-01
50%   -1.046374e-01 -3.573909e-01
75%    4.333115e-01 -2.424635e-02
max    3.891554e+00  9.667167e+00

Label Encoding on 'sex': {'female': np.int64(0), 'male': np.int64(1)}

Ordinal Encoding on 'class' (Third < Second < First):
    class  pclass_name_ordinal
0   Third                  0.0
1   First                  2.0
9  Second                  1.0

One-Hot Encoding on 'embarked':
    embarked_Q  embarked_S
0          0.0         1.0
1          0.0         0.0
2          0.0         1.0
3          0.0         1.0
4          0.0         1.0

======================================================================
Combined preprocessing pipeline (recommended approach for ML models)
======================================================================

Final preprocessed feature matrix (via Pipeline + ColumnTransformer):
        age      fare     sibsp     parch  sex_female  sex_male  embarked_C
embarked_Q  embarked_S  class_First  \
0 -0.565736 -0.502445  0.432793 -0.473674         0.0       1.0         0.0
0.0         1.0           0.0
1  0.663861  0.786845  0.432793 -0.473674         1.0       0.0         1.0
0.0         0.0           1.0
2 -0.258337 -0.488854 -0.474545 -0.473674         1.0       0.0         0.0
0.0         1.0           0.0
3  0.433312  0.420730  0.432793 -0.473674         1.0       0.0         0.0
0.0         1.0           1.0
4  0.433312 -0.486337 -0.474545 -0.473674         0.0       1.0         0.0
0.0         1.0           0.0

   class_Second  class_Third
0           0.0          1.0
1           0.0          0.0
2           0.0          1.0
3           0.0          0.0
4           0.0          1.0

All preprocessing steps completed successfully.
```